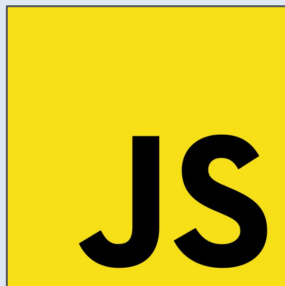




Institut TIC
de Barcelona



RA2.FE - Introducció a javascript. Sintaxis.

const

Quan el valor **no canvia** durant el procés d'execució.

Ex: *Any de naixement, PI, ...*

Sempre que puguem declarem un *const* abans que *let*.

S'ha de declarar sempre amb un valor.

- Si el valor és primitiu i directe (no calculat) la constant es declara amb majúscules.
- En els altres casos amb camelCase (*nomVariableExemple*).

let

Quan el valor **pot canviar** durant el procés d'execució.

Ex: *Comptadors, numero random,...*

Es declara sempre amb camelCase

var

Quan el valor **pot canviar** durant el procés d'execució.

Ex: *Comptadors, resultats d'operacions matemàtiques,...*

S'aconsella no utilitzar-la!!

```
<script>
  // Constants:
  // Son valors que no canviaran
  // Amb majúscula ja que s'assignen valors primitius
  const BIRTH_YEAR = 1991;
  const CURRENT_YEAR = 2024;

  // Let ja que el seu valor pot canviar.
  // El let es pot assignar sense valor.
  // Es defineix amb camlCase.
  let age;

  // Constant:
  // El seu valor no canvia.
  // Al no ser un valor primitiu
  // es defineix amb camlCase
  const calcYear = function () {
    age = CURRENT_YEAR - BIRTH_YEAR;
    console.log("La teva edat és " + age);
  };

  calcYear();
</script>
```

Regles nomenclatures

- Utilitzar els caràcters (0-9, a-z, A-Z) i el guió baix '_'
- No utilitzar ni números ni el guió baix ('_') a l'inici de la variable
- Utilitzar la convenció camelCase
- Case sensitive: *myage* != *myAge*
- No utilitzar paraules reservades: for, if,...

Perquè no utilitzar var?

Hoisting

Les variables declarades com a **var** (i les funcions declaratives **function** **nomFunction(){}**) son assignades en memòria en temps d'execució abans d'executar la resta de codi. **Això pot comportar molt errors i bugs al desenvolupament.**

- Utilitzar variables abans de declarar-les

```
//Puc cridar a una variable abans de declarar-la
//El seu valor per defecte és undefined.
console.log("El valor de la variable: " + comptador);

//Declaro la variable amb un valor.
var comptador = 3;
console.log("El valor de la variable: " + comptador);
```

El valor de la variable: undefined

El valor de la variable: 3

Perquè no utilitzar var?

- Problemes amb el block scope (condicionals, for, ...)

```
//En condicionals o bucles, si definim variables es mantenen fora del seu àmbit
if (true) {
  var x = 10;
  let y = 20;
}
console.log(x); // 10 (accessible fora del bloc!)
console.log(y); // ReferenceError: y is not defined
```

- Tornar a declarar una mateixa variable

```
var count = 1;
// Declarem una altra vegada la variable sense donar errors
var count = 2;
console.log(count); // 2
```

Perquè utilitzar *const* i *let*?

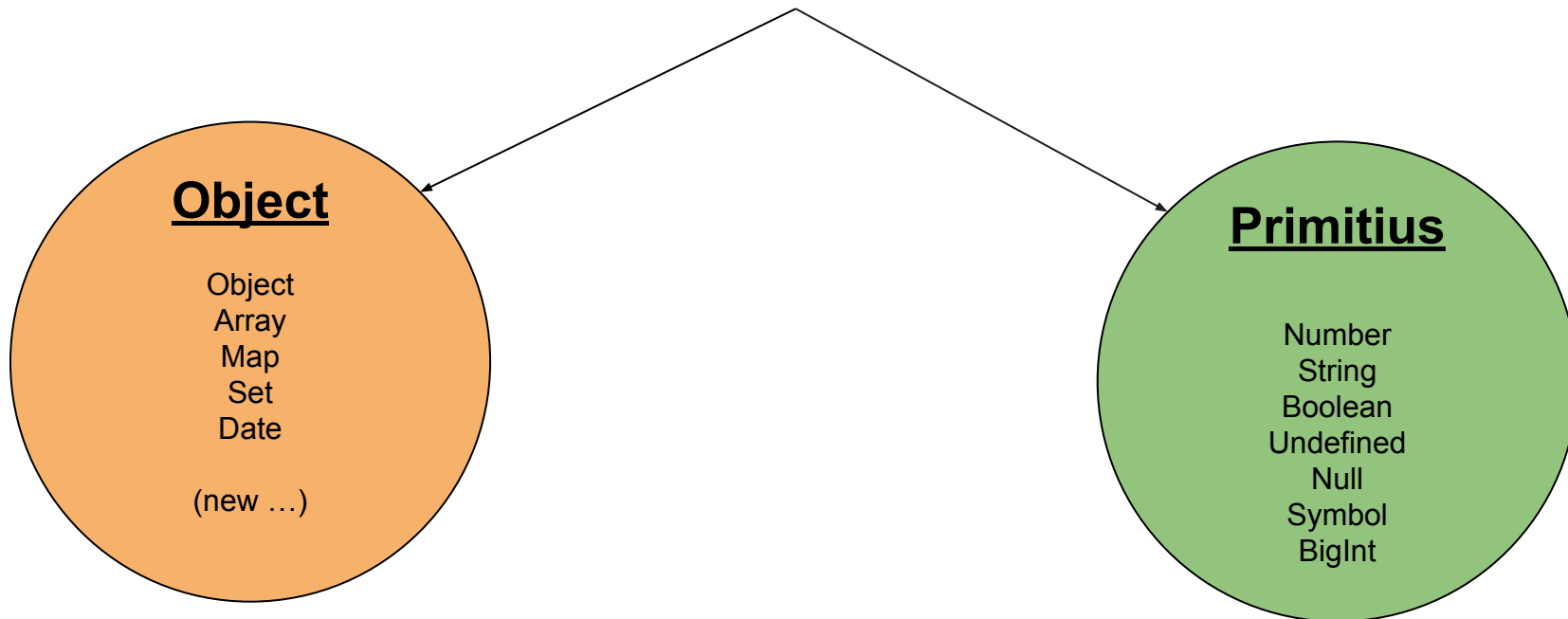
- **TDZ (Temporal dead zone)**

Les variables *let* i *const* tenen el mateix comportament, amb la diferència que si les utilitzem abans de declarar-les, ens dona un error de compilació que indica que la variable existeix però no es pot utilitzar fins que no s'inicialitzi.

```
//Amb el let i const no puc cridar a una variable abans de declarar-la  
//Indica que dona un error  
console.log("El valor de la variable: " + comptador);  
let comptador = 3;
```

! ▶ Uncaught ReferenceError: can't access lexical declaration 'comptador' before initialization
<anonymous> <http://127.0.0.1:5500/index.html:130>

Valor



Dades primitives

La forma més bàsica de representar una dada.

```
// NUMBER (sencer o decimal)
const num1 = 33;
const decimal1 = 5.25;
console.log(num1, typeof num1, decimal1, typeof decimal1);

// STRING (text)
const string1 = "Marta";
console.log(string1, typeof string1);

// BOOLEAN (true o false)
const boolean1 = true;
console.log(boolean1, typeof boolean1);

// UNDEFINED ( no té definit un valor)
let undefined1;
console.log(undefined1, typeof undefined1);

// NULL (té definit un valor però està buit)
let null1 = null;
console.log(null1, typeof null1);

// SYMBOL (Un tipus de dada les instàncies del qual són úniques i immutables)
const sym1 = Symbol();
console.log(sym1, typeof sym1);

// BIGINT (número sencer amb precisió arbitrària)
const begin1 = 992n;
console.log(begin1, typeof begin1);
```

33	number	5.25	number
Marta	string		
true	boolean		
undefined	undefined		
null	object		
Symbol()	symbol		
992n	bigint		

object ???

Dades primitives

typeof ens permet saber el tipus de valor que té una variable

```
const num1 = 33;  
console.log(num1, typeof num1);  
  
const num2 = "33";  
console.log(num2, typeof num2);
```

33	number
33	string



Compte amb la confusió amb els números definits com String

```
// JS és no tipat  
const num1 = 33;  
console.log(num1, typeof num1);  
  
num1 = "33";  
console.log(num1, typeof num1);
```

33	number
33	string



Recordeu que Js és un llenguatge no tipat i una variable pot canviar el tipus dinàmicament.

Tipus de dades - primitives

Number

```
//Tipus Primitiu
const num1 = 20;
console.log(num1, typeof num1);

const num2 = 20.2;
console.log(num2, typeof num2);

const num3 = 0.10213;
console.log(num3, typeof num3);

const num4 = -3;
console.log(num4, typeof num4);

const num5 = 3.125e7;
console.log(num5, typeof num5);

//Tipus objectes
const num6 = new Number("20");
console.log(num6, typeof num6);
```

```
20 number
20.2 number
0.10213 number
-3 number
31250000 number
► Number { 20 } object
```

```
//Convertir String a Number
const num1 = Number("20");
console.log(num1, typeof num1);

//Convertir Boolean a Number
const num2 = Number(true);
console.log(num2, typeof num2);

const num3 = Number(false);
console.log(num2, typeof num3);
```

```
20 number
1 number
0 number
```

String

```
//Tipus primitiu
const text1 = "Hello Javascript"; // cometes simples
const text2 = "Hello Javascript"; // cometes dobles

//Objecte
const text3 = String("Hello Javascript");
const text4 = new String("Hello Javascript");
```

Caràcters especials

Operador	Funció
\n	Salt de línia
\t	Tabulació horitzontal
\b	Retrocés
\\	Barra invertida
\'	Cometa simple
\"	Cometes dobles

String

Mètodes i atributs més importants:

- ***length***: ens retorna la llargada del String
- ***indexOf***: ens indicarà quina posició té el text en cas de trobar-lo. Si no apareix ens retornarà un -1
- ***charAt(i)***: mostra el caràcter de la posició
- ***includes***: ens retorna un true/false si el text indicat forma part del string donat
- ***trimStart()***: en el cas de tenir espais en blanc a l'inici d'un string els eliminaria
- ***trimEnd()***: en el cas de tenir espais en blanc al final d'un string els eliminaria
- ***replace()***: reemplaça un text d'un string per un altre. Primer posem el que ha de cercar i després el text reemplaçat
- ***slice()***: et permet crear una subcadena de la cadena principal donant el caràcter inicial i el final
- ***split()***: retorna un array amb les paraules d'una cadena separades per un símbol específic
- ***toUpperCase()/toLowerCase()***: converteix a majúscules/minúscules un text
- ***padStart()/padEnd()***: emplena la cadena actual amb un caràcter fins a una longitud indicada

Tipus de dades - primitives

String

Concatenar text amb variables:

- **concat:** concatena dos strings (ho podem fet també amb el +)

```
const product = "Portàtil";  
const price = 30.3;  
  
console.log(product.concat(price));  
console.log(product + " " + price);  
console.log("El teu producte és" + product + " té un cost de " + price);  
console.log("El teu Producte", product, " té un cost de ", price);
```

Portàtil30.3

Portàtil 30.3

El teu producte ésPortàtil té un cost de 30.3

El teu Producte Portàtil té un cost de 30.3

NOTA: Els tres primers casos converteix el
Number a *String*

- **template string:** forma més moderna de concatenar

```
const product = "Portàtil";  
const price = 30.3;  
  
console.log(`El teu producte és ${product} té un cost de ${price}`);
```

El teu producte és Portàtil té un cost de 30.3

Boolean

```
//Primitius
const bool1 = true;
const bool2 = false;
console.log(bool1, typeof bool1, bool2, typeof bool2);

//Objecte
const bool3 = new Boolean(true);
console.log(bool3, typeof bool3);
```

true boolean false boolean

► Boolean { true } object

Tipus de dades - primitives

Undefined

```
//Undefined: Declarem la variable però no té informat un valor  
let variable;  
console.log(variable, typeof variable);
```

undefined undefined

```
if(variable === undefined){  
  console.log("variable no definida");  
}
```

```
if(variable == undefined){  
  console.log("variable no definida");  
}
```

```
if(!variable){  
  console.log("variable no definida");  
}
```

variable no definida

variable no definida

variable no definida

Tipus de dades - primitives

Null

```
//NULL: Variable que està declarada però que pot no tenir un valor informat en algun moment.  
//Exemple: Una consulta a bases de dades (pot retornar informació o no retornar res)  
let variable = 3;  
variable = null;  
console.log(variable, typeof variable);
```

null object

```
if(variable === null){  
  console.log("variable sense valor")  
}  
if(variable == null){  
  console.log("variable sense valor")  
}  
if(!variable){  
  console.log("variable sense valor")  
}
```

variable sense valor
variable sense valor
variable sense valor

BigInt

```
//BigInt permet treballar amb nombres enters molt grans.  
const numNumber = 123456789123456789123456789;  
const numBigInt = 123456789123456789123456789n;  
  
console.log(numNumber, typeof numNumber);  
console.log(numBigInt, typeof numBigInt);
```

1.2345678912345679e+26 number

123456789123456791337762816n bigint

Object Date

Constructor	Paràmetre
<code>new Date();</code>	Si no se li passen paràmetres, el constructor agafarà la data del sistema.
<code>new Date(valor);</code>	Nombre sencer que representa el nombre de milisegons des de l'1 de gener de 1970 00:00:00 UTC (Època UNIX).
<code>new Date(dataString);</code>	Valor String que representa una data. La String ha d'estar en un format reconegut pel mètode <code>Date.parse()</code> - month/date/year (como: 6/13/2004) - month_name date, year (como: January 12, 2004) - day_of_week month_name date year hours:minutes:seconds time_zone (como: Tue May 25 2004 00:00:00 GMT-0700) - ISO 8601 YYYY-MM-DDTHH:mm:ss.sssZ (como: 2004-05- 25T00:00:00).

Aritmètics

Operador	Funció
+	Suma
-	Resta
*	Multiplicació
/	Divisió
%	Mòdul
++	Autoincrement
--	Autodecrement

Assignació

Operador	Funció
=	Assigna
+=	Suma i assigna
-=	Resta i assigna
*=	Multiplica i assigna
/=	Divideix i assigna
%=	Mòdul i assigna

```
let a = 100;
```

```
a += 10;           // 110
```

```
a -= 15;           // 95
```

```
a *= 10;           // 950
```

```
a /= 2;             // 475
```

```
a %= 2;             // 1 (el mòdul és 1, doncs  
                     475 és senar)
```

Autoincrementable / Autoincrementable

```
let cont = 5;
```

```
cont++;
```

// Incrementa el valor però fins a la següent crida no ho veurem reflectit

```
cont--;
```

// Decrementa el valor però fins a la següent crida no ho veurem reflectit

```
++cont;
```

// Incrementa el valor i ens el retorna incrementat

```
--cont;
```

// Decrementa el valor i ens el retorna decrementat

```
cont += 3;
```

// Ens retorna el valor amb un + 3 d'increment

```
cont -= 3;
```

// Ens retorna el valor amb un -3 de decrement

Comparació

Operador	Funció
>	Major que
>=	Major o igual que
<	Menor que
<=	Menor o igual que
!=	Diferent
==	Igual que
===	Estrictament igual (sense conversió de tipus)
!==	Estrictament diferent (sense conversió de tipus)

```
let result1 = ("55" == 55);    //true  
let result2 = ("55" === 55);  //false
```

```
let result1 = ("55" != 55);    //false  
let result2 = ("55" !== 55);  //true
```

Lògics

Operador	Funció
!=	Not
&&	AND
	OR

Operador ternari

Operador	Funció
? :	Condicional

```
variable = Condición ? Valor_cierto : Valor_falso;
```

```
let max = (num1 > num2) ? num1 : num2;
```


Valors *Truthy* / *Falsy*

En Javascript cada valor té un equivalent booleà (es pot convertir a *true* o *false*). És important saber-ho per les condicions ja que serà molt necessari per exemple per controlar si un valor està declarat, informat, ...

Valor	Descripció	Condicional que és Truthy (té valor correcte)	Exemple condicional Falsy (no té valor correcte)
<i>undefined</i>	Quan la variable no està definida.	<pre>let numProd = 1; // té definit un valor if(numProd) { } if (numProd !== undefined) { }</pre>	<pre>let numProd; if(numProd){ console.log("La variable està definida"); }else{ console.log("La variable NO està definida"); //Resultat }</pre>
0	Number no definit. Quan el valor d'una variable és 0.	<pre>let numProd = 1; //té un number diferent a 0 if (numProd) {} if (numProd !== 0) {}</pre>	<pre>let numProd = 0; if (numProd) { console.log("La variable està definida"); } else { console.log("La variable NO està definida"); //Resultat }</pre>
""	String no definit. Quan el valor d'una variable és ""	<pre>let nom = "Marta"; // té valor string if (nom) { } if (nom !== "") { }</pre>	<pre>let nom = ""; if (nom) { console.log("La variable està definida"); } else { console.log("La variable NO està definida"); //Resultat }</pre>

Valors *Truthy* / *Falsy*

Valor	Descripció	Condicional que és Truthy (té valor correcte)	Exemple condicional Falsy (no té valor correcte)
<i>Nan</i>	<i>Not a Number.</i> Quan una variable té definit un valor diferent a number però es tracta com si fos un number.	<i>const numProd = 1;</i> <i>if (!isNaN(numProd)) { }</i> <i>if (numProd > 0) { }</i>	<i>const numProduct = "Computer";</i> <i>if (numProduct > 0) {</i> <i>console.log("La variable és un number");</i> <i> } else {</i> <i>console.log("La variable NO és un number"); //Resultat</i> <i> }</i>
			<i>const numProduct = "Computer";</i> <i>if (!isNaN(numProduct)) {</i> <i>console.log("La variable és un number");</i> <i> } else {</i> <i>console.log("La variable NO és un number"); //Resultat</i> <i> }</i>
<i>null</i>	Quan tenim una variable definida però no té valor.	<i>const numProd = 3;</i> <i>if (numProd) { }</i> <i>if (numProd != null)</i>	<i>const numProd = null;</i> <i>if (numProd) {</i> <i>console.log("numProd té un valor");</i> <i> } else {</i> <i>console.log("numProd NO té valor"); //Resultat</i> <i> }</i>

Math

Funció	Definició	Exemple	Resultat
Math.PI	Ens retorna el número PI	<code>res = Math.PI;</code>	3.1415...
Math.round()	Arrodoneix a l'alça i és ≥ 5 sinó a la baixa	<code>res = Math.round(2.5);</code>	3
Math.ceil()	Arrodoneix sempre a l'alça	<code>res = Math.ceil(2.2);</code>	3
Math.floor()	Arrodoneix sempre a la baixa	<code>res = Math.floor(2.2);</code>	2
Math.sqrt()	Arrel quadrada	<code>res = Math.sqrt(144);</code>	12
Math.abs()	Valor absolut	<code>res = Math.abs(-300);</code>	300
Math.pow()	Potència	<code>res = Math.pow(8, 3);</code>	512
Math.min()	Ens retorna el valor mínim	<code>res = Math.min(4, 2, 1, -3);</code>	-3
Math.max()	Ens retorna el valor màxim	<code>res = Math.max(21, 4, 11, 5);</code>	21

Math random

Math.random() ens retornarà un valor aleatori decimal entre 0 i 1.

```
Math.random()
```

Si volem que sigui enter podem utilitzar Math.floor()

```
Math.floor(Math.random())
```

Math random

Si volem que tingui un valor màxim

```
Math.floor(Math.random() * valor)
```

Si volem que estigui entre dos valors

```
Math.floor(Math.random() * (max - min + 1) + min)
```

If

```
if(expresión){  
    instruccions;  
}
```

```
if(expresión){  
    instruccions_si_true;  
}else{  
    instruccions_si_false;  
}
```

Switch

```
switch(expresió){  
    case valor1:  
        instruccions;  
        break;  
    case valor2:  
        instruccions2;  
        break;  
    default:  
        instruccions else;  
}
```

For

```
for(valor_inicial; condició; increment_o_decrement){  
    instrucciones  
}
```

```
const total = 10;  
for (let i=0; i < total; i++){  
    alert(i);  
}
```


For-by

```
for(valor_inicial of collection){  
    //valor_inicial serà un article de la collection  
}
```

```
const animals =["gos", "gat", "cavall"];  
for (const animal of animals){  
    console.log(animal);  
}
```

While

```
while(expresión){  
    instrucciones  
}
```

```
let i = 0;  
while (i < 10) {  
    i += 2;  
}
```

```
do{  
    instrucciones  
} while(expresión)
```

```
let i = 0;  
do {  
    i += 2;  
} while (i < 10);
```

Break i continue

- *Break* provoca la interrupció de l'execució i salta a la primera línia després del búcle.
(surt del bucle)
- *Continue* provoca la interrupció de l'execució y salta a la següent iteració del búcle.
(surt de la iteració actual però no del bucle)

```
// Comentari d'una línia
```

```
/*
```

```
    Comentari de més  
    línies
```

```
*/
```